

AprilTag Machine Code Follow (No-PTZ)

1. Experiment Objective

The camera tracks an AprilTag marker, with body rotation tracking + distance following keeping the Tag centered in the frame and maintaining a constant distance. Distance following is based on the 3D pose Z value (actual depth), which is more accurate than pixel-size estimation.

2. Experiment Principle

1. Key Terms

Term	Description
AprilTag	A visual fiducial marker system, similar to a QR code but designed specifically for pose estimation; computes 3D position from the 4 corner points
3D Pose Estimation	Computes 6-DOF (x, y, z, roll, pitch, yaw) from the Tag corner coordinates + known physical size + the PnP algorithm
Distance Following (based on pose_z)	Directly uses the Z value of the 3D pose (actual depth in meters), far more accurate than pixel-size estimation
tag36h11	One of the AprilTag families, with 36-bit encoding + 11-bit error detection and high robustness
quad_decimate	Image downsampling factor; increasing it speeds up detection but reduces long-distance detection
QoS	best_effort (image input + cmd_vel downlink), depth=1

2. Technical Architecture

The distinctive feature of AprilTag following is that it uses the 3D pose Z value rather than pixel size for distance estimation, which is more accurate:

```
/camera/image_raw (BEST_EFFORT)
  → april_tag_tracker_node (enable_distance_follow:=true)
    └─ apriltag.Detector.detect() → 4 corner points + 3D pose (pose_z)
      └─ Body control (10 Hz):
        └─ Pixel error → PD angular velocity
          └─ pose_z error → PD linear velocity → /cmd_vel
```

3. Core Topics Quick Reference

Firmware Downlink

Topic	Message Type	QoS	Field	Range	Direction
/cmd_vel	geometry_msgs/Twist	best_effort + keep_last(1)	linear.x	-0.30~0.30 m/s	+ = forward, - = backward
			angular.z	-0.60~0.60 rad/s	+ = left turn, - = right turn

3. Experiment Steps

1. Hardware Preparation

Hardware	Requirement
PTZ Version	✗
No-PTZ Version	☑
Required Peripherals	ESP32 camera (mounted on bracket), a printed AprilTag marker (tag36h11 family)

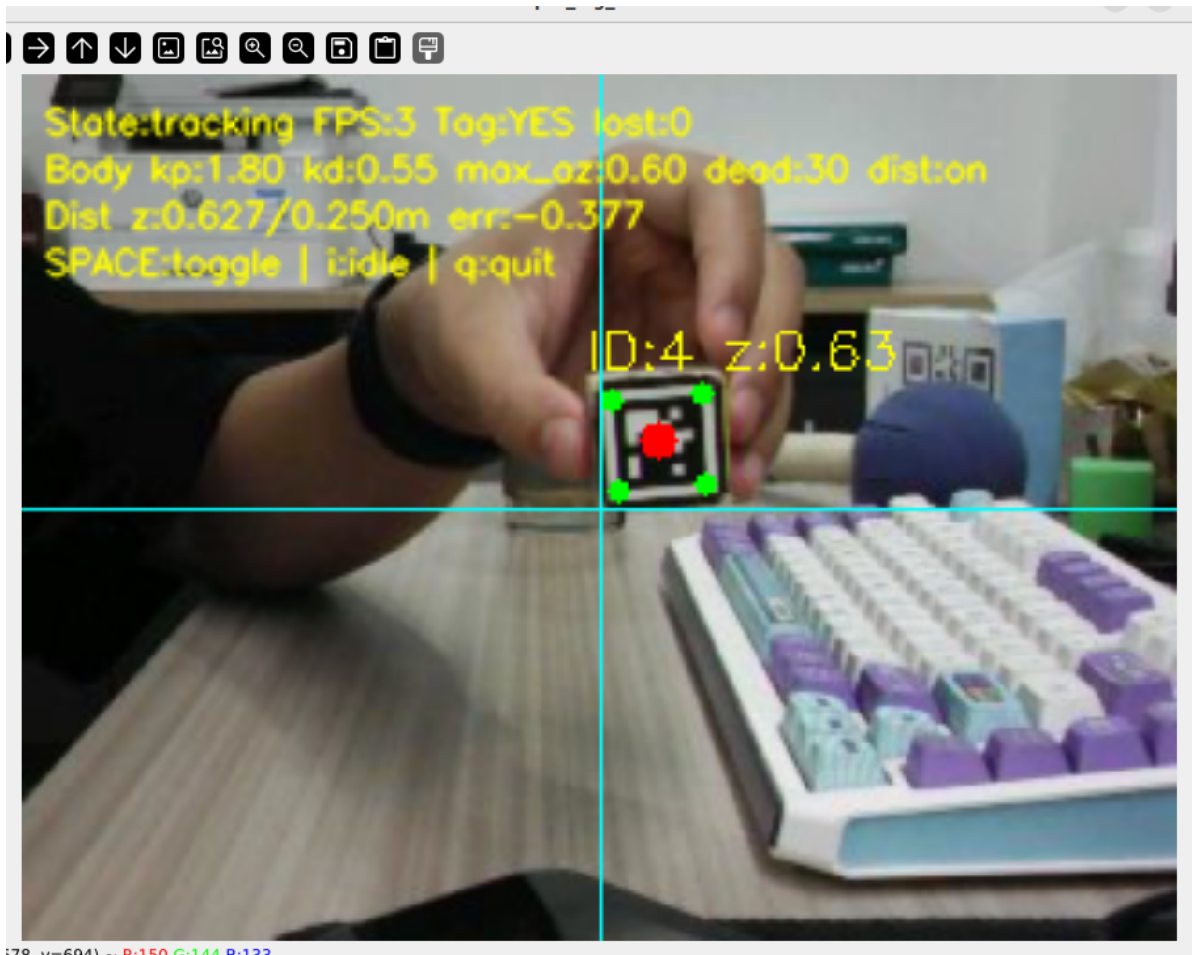
2. Start Agent + Camera + Joint Model (Terminal 1)

```
ros2 launch microros_v2_bringup car_base.launch.py
```

```
yahboom@yahboom:~$ ros2 launch microros_v2_bringup car_base.launch.py
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-07-28-15-04-46-269642-yahboom-579370
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [micro_ros_agent-1]: process started with pid [579411]
[INFO] [camera_driver-2]: process started with pid [579413]
[INFO] [robot_state_publisher-3]: process started with pid [579415]
[INFO] [joint_state_publisher-4]: process started with pid [579417]
[INFO] [ekf_node-5]: process started with pid [579419]
[micro_ros_agent-1] [1785222287.404236] info      | UDPv4AgentLinux.cpp | init          | running...    | port: 8090
[robot_state_publisher-3] [INFO] [1785222288.847816851] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-3] [INFO] [1785222288.848013042] [robot_state_publisher]: got segment base_link
[robot_state_publisher-3] [INFO] [1785222288.848028942] [robot_state_publisher]: got segment bwheel_Link
[robot_state_publisher-3] [INFO] [1785222288.848037122] [robot_state_publisher]: got segment cam1_Link
[robot_state_publisher-3] [INFO] [1785222288.848043874] [robot_state_publisher]: got segment cam2_Link
[robot_state_publisher-3] [INFO] [1785222288.848050538] [robot_state_publisher]: got segment cam3_Link
[robot_state_publisher-3] [INFO] [1785222288.848057264] [robot_state_publisher]: got segment imu_frame
[robot_state_publisher-3] [INFO] [1785222288.848063747] [robot_state_publisher]: got segment laser_frame
[robot_state_publisher-3] [INFO] [1785222288.848070240] [robot_state_publisher]: got segment lfwheel_Link
[robot_state_publisher-3] [INFO] [1785222288.848076888] [robot_state_publisher]: got segment rfwheel_Link
[joint_state_publisher-4] [INFO] [1785222289.372349515] [joint_state_publisher]: Waiting for robot_description to be published on the robot_description
[camera_driver-2] [INFO] [1785222289.716161193] [esp32_ip_camera_publisher]: camera node started, camera_url: http://192.168.12.37:81/stream
[camera_driver-2] [WARN] [1785222292.705741414] [esp32_ip_camera_publisher]: Camera not opened, trying reconnect... (next retry in 1.0s)
[camera_driver-2] [INFO] [1785222294.072701944] [esp32_ip_camera_publisher]: IP camera stream opened successfully
```

3. Start AprilTag Follow (Terminal 2)

```
ros2 launch microros_v2_no_ptz_visual_control_pkg april_tag_follow.launch.py
```



5. Operation Instructions

Key	Function
Space	Toggle follow/standby
q / ESC	Exit

Distance Follow Behavior: Tag approaches (pose_z decreases) → move backward; Tag moves away (pose_z increases) → move forward; depth within tolerance (± 0.02 m) → stay still.

4. Experiment Result

- Tag approaches → the car moves backward to keep the distance (based on the 3D depth Z value)
- Tag moves away → the car moves forward to follow
- Tag distance within the desired range (0.25 m) → the car stays still
- The terminal outputs the Tag 3D pose (x, y, z, roll, pitch, yaw) in real time
- Tag lost → the car automatically stops once the lost-target count exceeds the threshold

5. Code Explanation

Source code paths:

- Launch:

```
~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/launch/april_tag_follow.launch.py
```

- Node source:

```
~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz_visual_control_pkg/april_tag_tracker_node.py
```

1. Core Node Explanation

`april_tag_follow.launch.py` launches `april_tag_tracker_node`, with `enable_distance_follow` defaulting to `true`. Unlike other follow modes, AprilTag distance following directly uses the 3D pose Z value:

```
# Source file:
~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz_visual_control_pkg/april_tag_tracker_node.py

# Distance follow: based on the 3D depth Z value (not pixel size)
if self._dist_follow and tz is not None and tz > 0:
    err_z = self._target_z_m - tz # target_z unit: meters
    if abs(err_z) > self._dist_tol_z:
        lx = self._dist_kp * err_z
        lx = clamp(lx, -self._max_lx, self._max_lx)
        if self._reverse_linear:
            lx = -lx
        cmd.linear.x = lx
```

Key Advantage: Accurate distance is obtained without calibrating the actual target size. The `tag_size` parameter (default 0.048 m) must match the physical side length of the printed Tag, otherwise the 3D pose will be inaccurate.

2. Key Parameters

Parameter definition files:

- Launch:

```
~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/launch/april_tag_follow.launch.py
```

- Node:

```
~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz_visual_control_pkg/april_tag_tracker_node.py
```

Parameter	Type	Default	Description
<code>tag_family</code>	str	<code>tag36h11</code>	AprilTag family
<code>tag_size</code>	float	<code>0.048</code> m	Tag physical side length (affects 3D pose accuracy)
<code>quad_decimate</code>	float	<code>2.0</code>	Image downsampling factor
<code>tag_select_strategy</code>	str	<code>largest</code>	Multi-Tag selection strategy
<code>deadband_x_pixel</code>	float	<code>30.0</code> px	Horizontal deadband
<code>angular_kp</code> / <code>angular_kd</code>	float	<code>1.8</code> / <code>0.55</code>	Angle PD gains
<code>max_angular_z</code>	float	<code>0.6</code> rad/s	Maximum turning speed
<code>enable_distance_follow</code>	bool	<code>true</code>	Enable distance following
<code>target_tag_pose_z</code>	float	<code>0.25</code> m	Desired following distance (based on 3D pose_z)
<code>distance_tolerance_pose_z</code>	float	<code>0.02</code> m	Distance tolerance deadband
<code>distance_kp</code> / <code>distance_kd</code>	float	<code>1.5</code> / <code>0.03</code>	Distance PD gains
<code>max_linear_x</code>	float	<code>0.30</code> m/s	Maximum forward/backward speed

3. Node Description

Node	Package	Description
<code>april_tag_tracker_node</code>	<code>microros_v2_no_ptz_visual_control_pkg</code>	AprilTag 6-DoF tracking + following, 3D depth distance following

6. Common Issues

Issue	Cause	Solution
Inaccurate distance	The Tag size parameter <code>tag_size</code> does not match the actual size	Precisely measure and set the Tag physical side length
Tag not detected	Tag too small, reflective, or <code>quad_decimate</code> too large	Enlarge the Tag and lower <code>quad_decimate</code>
Pose jitter	Lighting changes or the Tag tilted too much	Keep the Tag flat and the lighting uniform
Slow following response	Unreasonable distance PD parameters	Adjust <code>distance_kp</code> / <code>target_tag_pose_z</code>

